BITÁCORA 1

Fibonacci ingenuo frente a iterativo

Universidad Nacional Experimental de Guayana (UNEG)

Análisis y Diseño de Algoritmos · Prof. Ana Sosa

Equipo 6

Integrantes: Abraham Salazar, Anthony Aular, Aaron Urbina, Sigles Velasquez, María Carolina Brito, Angel Parejo

6 de noviembre de 2026

1. Especificación

El problema es calcular el n-ésimo número de Fibonacci, y las dos soluciones que aquí se comparan parten del mismo contrato: la precondición exige que n sea un entero mayor o igual que 0. La postcondición exige que el valor devuelto sea F(n), definido por F(0) = 0, F(1) = 1 y F(n) = F(n-1) + F(n-2) para todo n mayor o igual que 2. Quedan fuera del contrato los negativos, los reales, los textos y los booleanos. En Python, bool es subclase de int, así que True y False pasarían un chequeo que solo usara isinstance(n, int). La especificación los rechaza de forma explícita: un valor de verdad no es un índice de la sucesión.

Fibonacci ingenuo traduce la recurrencia de manera directa: para n mayor que 1 hace dos llamadas, con n-1 y con n-2, y suma los resultados. Fibonacci iterativo conserva únicamente los dos últimos valores y recorre i desde 2 hasta n. Los dos cumplen la misma postcondición, y difieren solo en el trabajo que repiten y en el orden de crecimiento, que es el objeto de esta bitácora.

2. Corrección

La versión ingenua es correcta por inducción sobre n. Si n es 0 o 1, el valor devuelto es el de la definición. Si n es mayor que 1 y las llamadas con argumento menor ya devuelven el Fibonacci correspondiente, la suma es F(n-1) + F(n-2), que es F(n). La recursión termina porque cada llamada recibe un entero estrictamente menor y la precondición impide bajar de 0.

La versión iterativa se justifica con un invariante de ciclo. Justo antes de ejecutar la iteración i, con i entre 2 y n, se cumple a = F(i-2) y b = F(i-1). Inicialización: antes de entrar, a = 0 = F(0) y b = 1 = F(1), que es el invariante cuando i = 2. Mantenimiento: el paso sustituye a por b y b por a + b. El nuevo a es F(i-1) y el nuevo b es F(i-2) + F(i-1) = F(i). Eso es el invariante de la iteración i+1. Terminación: el rango es finito. Al acabar la vuelta i = n, b = F(n), que es la postcondición. Los casos n = 0 y n = 1 se demuestran aparte y no entran al ciclo.

La prueba test_invariante_del_ciclo revisa esa afirmación en cada vuelta para n = 30, reconstruyendo la sucesión con la misma recurrencia de la especificación. La prueba diferencial compara las dos funciones desde n = 0 hasta n = 20. Coincidir no reemplaza el invariante: lo contrasta con una segunda implementación.

3. Costo

En el ingenuo, el tiempo satisface T(0) = T(1) = Θ(1) y T(n) = T(n-1) + T(n-2) + Θ(1). La solución es T(n) = $\Theta(\phi^n)$, con $\phi = (1 + \sqrt{5}) / 2 \approx 1.618$. El número de llamadas es $2 \cdot F(n+1) - 1$. Cada llamada vuelve a calcular subproblemas enteros, sin guardar resultados.

En el iterativo hay un ciclo de n-1 vueltas. Contando cada suma como operación de costo constante, el tiempo es Θ(n). Ese modelo, sin embargo, se queda corto en Python, donde los enteros no tienen tamaño fijo y crecen con el valor que representan, de modo que sumar dos números grandes no cuesta lo mismo que sumar dos pequeños. F(n) tiene Θ(n) bits, porque crece como $\phi^n / \sqrt{5}$ y el número de bits es el logaritmo de ese valor. La suma del paso i opera cantidades de Θ(i) bits y cuesta Θ(i). Sumar esos costos desde i = 2 hasta n da $\Theta(n^2)$.

La razón del experimento es lo que permite separar un modelo del otro: si T(n) crece como $a \cdot n^k$, entonces T(2n)/T(n) tiende a 2^k. Para un costo lineal, k = 1 y la razón tiende a 2. Para un costo cuadrático, k = 2 y la razón tiende a 4. El grado observado se estima como $\log_2$ de la razón. Si el orden es $\Theta(\phi^n)$, la razón T(2n)/T(n) crece como ϕ^n y no tiende a ninguna constante. Por eso el mismo cociente sirve para los dos algoritmos, y la lectura es distinta.

4. Implementación y pruebas

El código de entrega está en fibonacci.py, donde validar_n aplica la precondición y las dos funciones, fibonacci_ingenuo y fibonacci_iterativo, traducen respectivamente la recurrencia y el ciclo; en esta última, el comentario del cuerpo enuncia el invariante justo en el punto en que se exige. Las pruebas van en pruebas.py, escritas con unittest, y no requieren instalar nada para ejecutarlas.

Las pruebas cubren seis obligaciones, agrupadas en cinco bloques: los casos conocidos se comprueban por separado en cada una de las dos funciones. Los casos conocidos comparan las dos funciones con F(0) hasta F(16), tomados de la definición. Los valores grandes comprueban la función iterativa real con F(50), F(100) y F(200), tomados de la sucesión publicada (OEIS A000045), y el ingenuo con F(25). La precondición se ensaya en las dos funciones con -1, -5, 1.5, 3.0, el texto "5", True y False. El contraste diferencial exige igualdad en $0 \le n \le 20$. El invariante se comprueba vuelta por vuelta hasta n = 30, sobre una réplica del ciclo; por eso los valores grandes verifican además la función entregada.

La corrida de esta máquina quedó en OK: 6 pruebas, 0 fallos y 0 errores. Comando: .\.venv\Scripts\python.exe -m unittest pruebas.py -v

5. Experimento

El diseño duplica el tamaño, que es la forma de leer T(2n)/T(n). Para cada n se hace una llamada de calentamiento, que no entra en la cuenta, y después cinco mediciones con time.perf_counter. De cada fila se resume la mediana, y también se guardan el mínimo y el máximo en el CSV. La semilla 20261106 se fija con random.seed y se escribe en los metadatos. No hay muestreo aleatorio: las dos funciones son deterministas y la semilla no mueve los tiempos.

Máquina de esta corrida: Windows-11-10.0.26200-SP0. 3.13.

Tamaños iterativos: [10000, 20000, 40000, 80000, 160000, 320000]. La razón de una fila es el tiempo mediano de n dividido por el tiempo mediano de n/2, cuando esa mitad también está en la serie. En

esta serie cada tamaño, salvo el primero, es el doble del anterior, así que cada cociente es exactamente T(2n)/T(n) del tamaño previo.

Las razones T(2n)/T(n) del iterativo (Tabla 1), en el orden de los tamaños, fueron 3.32, 4.19, 3.20, 3.71, 4.16. La razón media es 3.72 y el grado estimado, $\log_2$ de la media, es 1.89. Se usa la media y no solo la última razón porque cada cociente individual oscila con el ruido del reloj (aquí entre 3.20 y 4.19). El modelo de sumas a costo constante predice 2 y el modelo de bits predice 4. La medición favorece el costo cuadrático: F(n) tiene Θ(n) bits y sumarlos no es una operación de costo fijo.

Tabla 1. Fibonacci iterativo: mediana de cinco mediciones por tamaño.

n	Mediana (s)	T(2n)/T(n)	Grado $\log_2$
10000	0.001998	N/A	N/A
20000	0.006628	3.32	1.73
40000	0.027766	4.19	2.07
80000	0.088907	3.20	1.68
160000	0.330239	3.71	1.89
320000	1.373049	4.16	2.06

Fuente: datos/iterativo.csv, generado por experimento.py. N/A: el tamaño no tiene su mitad en la serie.

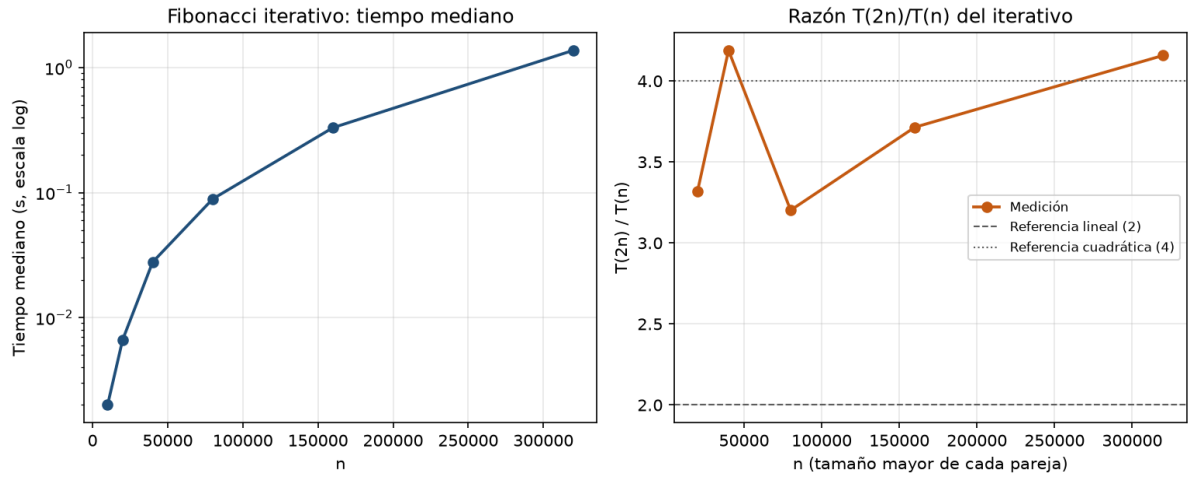


Figura 1. A la izquierda, tiempo mediano del Fibonacci iterativo. A la derecha, la razón T(2n)/T(n). La línea discontinua marca 2 (costo lineal si cada suma cuesta lo mismo) y la línea punteada marca 4 (costo cuadrático de bits).

Los archivos de la corrida son datos/iterativo.csv, datos/ingenuo.csv y datos/metadatos.txt. Las figuras quedan en figuras/. Volver a generarlos es ejecutar experimento.py con el mismo comando del README.

6. Contraste

Los dos algoritmos cumplen la misma especificación y no cuestan lo mismo. El ingenuo repite un árbol de llamadas de orden $\Theta(\phi^n)$. El iterativo hace una sola pasada, de manera que en la práctica el ingenuo deja de ser utilizable cerca de n = 40, porque

cada incremento de 1 multiplica el trabajo por ϕ. El iterativo se midió hasta n = 320000 en esta máquina.

Tamaños ingenuos: [8, 10, 12, 14, 16, 20, 24, 28, 32]. Se eligieron pares cuya mitad también está en la lista, para poder formar T(2n)/T(n). No se usaron los tamaños del iterativo: la recursión ingenua

tiene profundidad n, y con n = 10000 Python aborta con RecursionError antes de llegar al problema del tiempo. El contraste se hace por el orden, no por cronometrar los dos algoritmos en el mismo n.

Las razones del ingenuo (Tabla 2) van de 46.21 a 2538.2. No se acercan a 2 ni a 4: crecen al duplicar el argumento. Si T(n) es proporcional a ϕ^n, entonces T(2n)/T(n) es proporcional a $\phi^{n/2}$, donde n es el tamaño mayor de la pareja. El cociente entre cada razón medida y esa referencia quedó, en promedio, en 1.05. El experimento reproduce el orden exponencial: la referencia y la medición van juntas, con la holgura propia del reloj y del costo fijo de cada llamada. La pareja n = 28 se aparta: su razón 532.5 es 0.63 veces la referencia 843.0. La causa está en la medición de n = 14: su mediana (4.243e-04 s) supera en 1.7 veces a su propio mínimo (2.491e-04 s). Es ruido del reloj, no un cambio de orden.

Tabla 2. Fibonacci ingenuo: mediana por tamaño y referencia exponencial.

n	Mediana (s)	T(2n)/T(n)	Referencia $\phi^{n/2}$
8	1.400e-05	N/A	N/A
10	3.630e-05	N/A	N/A
12	9.480e-05	N/A	N/A
14	4.243e-04	N/A	N/A
16	6.469e-04	46.21	46.98
20	0.005358	147.60	122.99
24	0.038805	409.33	322.00
28	0.225944	532.51	843.00
32	1.641950	2538.18	2207.00

Fuente: datos/ingenuo.csv, generado por experimento.py. N/A: el tamaño no tiene su mitad en la serie.

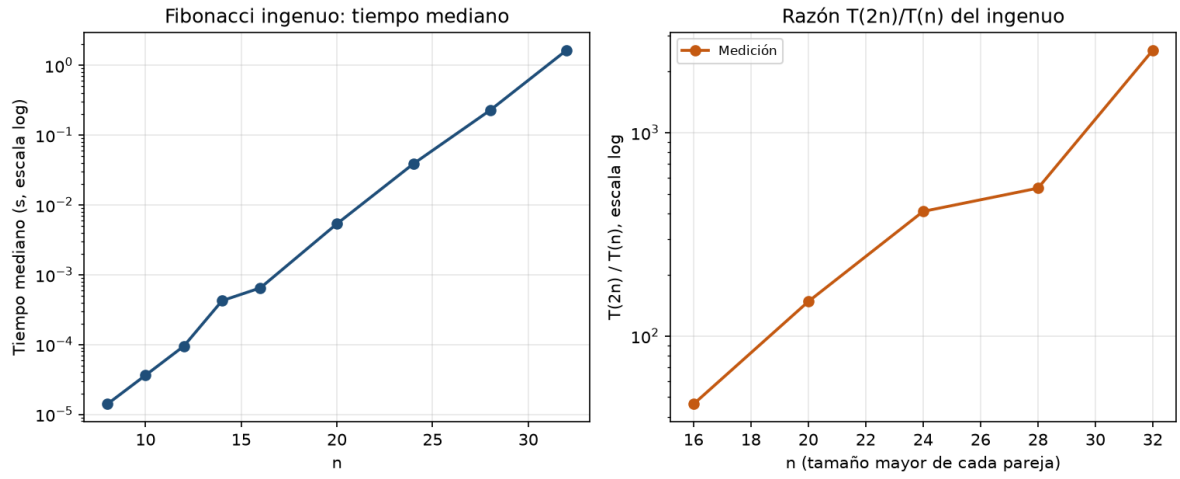


Figura 2. A la izquierda, tiempo mediano del Fibonacci ingenuo. La escala es logarítmica cuando los tiempos se separan en más de un orden. A la derecha, T(2n)/T(n). En un orden exponencial esta razón crece; en un orden polinomial tendería a una constante.

El contraste, entonces, queda como sigue: el ingenuo muestra una razón que crece con n, en la dirección de $\phi^{n/2}$. El iterativo queda entre 3.20 y 4.19: por encima de la referencia lineal 2 y alrededor de la referencia cuadrática 4. Esa lectura explica por qué no basta con decir que el ciclo es de n sumas. Para n grande, el algoritmo utilizable es el iterativo, con orden $\Theta(n^2)$ en el modelo de bits del lenguaje.

7. Uso de IA y reproducibilidad

Herramientas, versiones y fechas de uso: Google Gemini 3.5 Flas-lite y OpenAI ChatGPT GPT-6 de OpenAI, usados el 3, 4 y 5 de noviembre de 2026 por María Carolina Brito en la etapa de estudio previo; Cursor con el agente Auto, enrutador que selecciona el modelo automáticamente y por tanto no deja constancia de cuál respondió en cada turno, usado el 06/11/2026; y Claude Code 2.1.319 con el modelo Claude Opus 5.5 (claude-opus-5-5), usado el 06/11/2026; DeepSeek-V4.1-Flash. El informe se generó con fpdf2 2.9.0 y las gráficas con matplotlib 3.11.2, sobre Python 3.13.0 (Windows-11-10.0.26200-SP0).

En la etapa inicial, los días 3, 4 y 5 de noviembre de 2026, María Carolina Brito usó Gemini y ChatGPT para entender el Fibonacci recursivo e iterativo, y probó ambas versiones en consola antes de que el algoritmo se pasara a módulos.

Claude Code para preparar el entorno de Linux con las mismas versiones y añadir la prueba de valores grandes. También asistió a Angel Parejo en la preparación y verificación de los comandos, tanto los de la reproducción en Linux como los del repositorio. Calculó además el grado como $\log_2$ de la media de las razones; el estadístico lo eligió el equipo, y la sección 5 explica por qué, dado que cada cociente individual oscila entre 3.20 y 4.19 por ruido del reloj.

Qué se hizo con ellas: con Cursor y su agente Auto, el equipo convirtió el algoritmo, que estaba en una transcripción de consola, en módulos que se pueden ejecutar; añadió la validación de la precondición, las pruebas del invariante y el experimento de duplicación; generó los CSV y las dos gráficas; y redactó el borrador del informe. El texto corrido de las secciones 1 a 6 lo redactó esa herramienta y vive como cadenas de texto dentro de generar_informe.py, que es el programa que arma este PDF. El equipo preparo el texto, a partir del algoritmo probado en consola, del contrato de las dos funciones y de sus decisiones de método.

Qué verificamos y quién: Anthony Aular y Sigles Velasquez ejecutaron la suite en Windows 11 y confirmaron las 6 pruebas en OK; Angel Parejo repitió el experimento en Linux/WSL2 con las mismas versiones y comprobó que las razones coinciden: 3.72 en Windows frente a 3.66 en Linux; Angel Parejo estableció que el valor de n = 28 en Windows, 0.63 de la referencia, era ruido de medición, porque en Linux la misma pareja dio 1.09; Angel Parejo contrastó que cada cifra de las tablas y de las gráficas sale de los CSV y no se tecleó a mano. Ningún tiempo se tecleó a mano: todos salen de los CSV que escribe experimento.py, los de esta máquina en las secciones 5 y 6 y los de datos/linux/ en la Tabla 3. El comentario que cierra la sección 7.1 lo redactó Angel Parejo; Claude Code le hizo observaciones que él incorporó.

Se empleó la herramienta de Inteligencia Artificial exclusivamente como soporte para sintetizar, formatear y estructurar las reflexiones analíticas de los cuatro integrantes según el límite de extensión solicitado, manteniendo de forma íntegra e inalterada la información, los datos numéricos y las conclusiones técnicas presentadas en el documento original.

Qué se descartó: usar la transcripción con prompts >>> como evidencia, porque ese archivo no es un programa; concluir que el iterativo es lineal en tiempo de pared solo porque el ciclo da n vueltas.
Reproducir la corrida, desde la carpeta del proyecto:

C:\laragon\bin\python\python-3.13\python.exe -m venv .venv .\.venv\Scripts\python.exe -m pip install -r requirements.txt .\.venv\Scripts\python.exe -m unittest pruebas.py -v

.\.venv\Scripts\python.exe experimento.py .\.venv\Scripts\python.exe generar_informe.py # En Linux o macOS:

python3 -m venv .venv .venv/bin/python -m pip install -r requirements.txt .venv/bin/python -m unittest pruebas.py -v .venv/bin/python experimento.py .venv/bin/python generar_informe.py

Semilla: 20261106. El experimento cronometra funciones deterministas. La semilla se fija y se registra; no interviene en los tiempos. Reloj: time.perf_counter. Estadístico: mediana. 1 llamada por cada n, fuera de la mediana. Repeticiones: 5. Otra computadora va a obtener otros segundos y unas razones parecidas, no una copia bit a bit del reloj. El PDF se vuelve a armar con generar_informe.py y queda en informe/Equipo_6_Bitacora1_PRELIMINAR.pdf; el fichero entregado es esa salida tras una pasada de compresión, de modo que el contenido es idéntico aunque no sea byte a byte igual.

El repositorio público https://github.com/angelparejo/uneg-equipo6-bitacora1-fibonacci contiene fibonacci.py, pruebas.py, experimento.py, generar_informe.py, el README con los comandos, las versiones y la semilla, y las carpetas datos y figuras.

7.1 Reproducción en Linux El experimento se repitió sin cambios en otra máquina:

Linux-5.15.167.4-microsoft-standard-WSL2-x86_64-with-glibc2.39, Intel(R) Core(TM) i5-8265U CPU @ 1.60GHz (WSL2), Python 3.13.0, con la misma semilla, repeticiones, tamaños y versiones de matplotlib y fpdf2. Los datos están en datos/linux/ y las figuras en figuras/linux/.

Tabla 3. Misma corrida en dos entornos de ejecución.

Indicador	Windows	Linux
Iterativo: razón media	3.72	3.66
Iterativo: grado $\log_2$	1.89	1.87
Ingenuo n = 16: medida / $\phi^{n/2}$	0.98	0.97
Ingenuo n = 20: medida / $\phi^{n/2}$	1.20	1.01
Ingenuo n = 24: medida / $\phi^{n/2}$	1.27	1.08
Ingenuo n = 28: medida / $\phi^{n/2}$	0.63	1.09
Ingenuo n = 32: medida / $\phi^{n/2}$	1.15	1.12

Fuente: datos/ para Windows y datos/linux/ para Linux. Misma semilla, mismas repeticiones y mismos tamaños.

La reproducción en Linux la ejecutó Angel Parejo, con Claude Code como asistente en la preparación del entorno y en la preparación y verificación de los comandos, con las mismas versiones de Python y dependencias, para aislar discrepancias asociadas al entorno de ejecución. Se corrió sobre WSL2, es decir otro entorno sobre el mismo equipo, de modo que descarta efectos del sistema operativo pero no del hardware. El algoritmo iterativo mostró una razón media de 3.66, consistente con el costo cuadrático $\Theta(n^2)$, porque el costo de sumar crece con la longitud en bits de F(n). Por otro lado, el algoritmo ingenuo en n = 28 registró una relación de 1.09 con la referencia $\phi^{n/2}$, frente al 0.63 atípico de Windows, donde la mediana de n = 14 fue 1.7 veces su propio mínimo, lo que indica ruido de medición y no un comportamiento asintótico anómalo. Aunque los tiempos absolutos difieren (1.07 s en Linux frente a 1.37 s en Windows, para n = 320000), las razones coinciden: la constante depende de la máquina y el orden de crecimiento se mantiene. A la vista de ese ruido, el equipo propone aumentar las repeticiones en la próxima bitácora.

8. Declaración de autoría y responsabilidad

Son del equipo el algoritmo probado en consola y el contrato de las dos funciones, con el rechazo explícito de los booleanos. También el diseño del experimento de duplicación, la decisión de estimar el grado con la media de las cinco razones y no con la última, y la lectura de las razones T(2n)/T(n). El texto corrido de las secciones 1 a 6 lo redactó Cursor a partir de ese material, según se detalla en la Sección 7. Eso incluye la demostración por inducción, el enunciado del invariante de ciclo y el modelo de costo en bits: el equipo responde de ellos, pero no los escribió. Los seis integrantes leímos la versión final, verificamos lo indicado en la Sección 7 y respondemos por todo el contenido, incluidos los errores. Las transcripciones de las sesiones con las herramientas quedan a disposición de la profesora si las solicita.

9. Contribución de cada integrante

Abraham Salazar, transcripción original del algoritmo, con las versiones recursiva e iterativa probadas en consola, en colaboración con Maria Carolina Brito; acordó con Aaron Urbina el contrato de las dos funciones —precondición, postcondición y el rechazo explícito de los booleanos— y lo validó contra la implementación; Anthony Aular, corrida del experimento en Windows 11 con Python 3.13.0, de la que salen los datos y las gráficas que sostienen el informe, junto con Sigles Velasquez; Aaron Urbina,

acordó con Abraham Salazar el contrato de las dos funciones —precondición, postcondición y el rechazo explícito de los booleanos— y lo validó contra la implementación; Sigles Velasquez, corrida del experimento en Windows 11 con Python 3.13.0, de la que salen los datos y las gráficas que sostienen el informe, junto con Anthony Aular; Maria Carolina Brito, pruebas iniciales en consola para entender el Fibonacci recursivo e iterativo, los días 3, 4 y 5 de noviembre de 2026, apoyada en Gemini y ChatGPT (ver sección 7); colaboración con Abraham Salazar en la transcripción original; revisión del borrador del informe; Angel Parejo, creación y administración del repositorio Git y reproducción del experimento en Linux/WSL2 con Python 3.13.0, ambas con Claude Code como asistente en la preparación y verificación de los comandos; revisión crítica del análisis (valor atípico en n = 14, grado estimado con la media de las razones, prueba con F(50), F(100) y F(200)); redacción del comentario de la reproducción en Linux. El mensaje del foro repite esta misma lista sin cambios.

10. Referencias

10. Referencias Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). Introduction to Algorithms (4.ª ed.). MIT Press. ISBN 978-0-262-04630-5. — Invariantes de ciclo con el esquema de inicialización, mantenimiento y terminación que sigue la Sección 2, y notación asintótica.

Knuth, D. E. (1997). The Art of Computer Programming, Volume 1: Fundamental Algorithms (3.ª ed.). Addison-Wesley. ISBN 0-201-89683-4. — Sección 1.2.8, números de Fibonacci: la recurrencia, la forma cerrada $F(n) = \phi^n / \sqrt{5}$ redondeada y el conteo de llamadas del cálculo recursivo.

Knuth, D. E. (1998). The Art of Computer Programming, Volume 2: Seminumerical Algorithms (3.ª ed.). Addison-Wesley. ISBN 0-201-89684-2. — Sección 4.3.1, aritmética multiprecisión clásica: sumar dos enteros de i bits cuesta $\Theta(i)$ y no $\Theta(1)$, que es la premisa del modelo de bits de la Sección 3.